Recovery and demo runbook

Group 15 • Final assessment preparation • 3 November 2026

The 3 November audit found zero containers after the disk cleanup. Named PostgreSQL, Kafka, and monitoring volumes still exist, but service recovery and data integrity remain unverified. Source files match the October snapshot and 42 unit tests pass. The core stack must be restored before a live demo can satisfy the rubric.

The commands below are recovery instructions for the team. No containers were recreated, images rebuilt, volumes deleted, or load generators started while preparing these reports.

What the cleanup changed

Docker kill stops running containers abruptly. Docker rm removes containers and their writable layers. Docker rmi removes images or image references; it does not remove source files or ordinary named volumes. An abruptly stopped PostgreSQL instance may need its normal crash recovery when reopened.

The commands listed by the user did not request volume deletion. The audit confirmed the named volumes exist; it did not open the database and prove that all rows survived. Some untagged runtime/build artifacts and build cache remain, but the normal application image tags are absent.

Retained volume	Docker-reported size at inspection	Role
cec-temperature_postgres-data	1.653 GB	Application experiment state, history, checkpoints, outbox
cec-temperature_course-db-data	49.08 MB	Local instructor ground truth and accepted notifications
cec-temperature_kafka-data	2.14 GB	Local Kafka state and retained events/documents
cec-temperature_prometheus-data	87.92 MB	Operational metrics
cec-temperature_course-prometheus-data	655.2 MB	Local grading metrics
cec-temperature_grafana-data	52.43 MB	Operational Grafana state
cec-temperature_course-grafana-data	52.59 MB	Local grading Grafana state

Docker reported about 1.422 GB of build cache and a large untagged build image. “Reclaimable” means unused by a current container, not unimportant. Since all containers were removed, even the application database volume appears unused.

Step 1 Preserve data and inspect capacity

From the repository root:

```
df -h .
sudo -n docker system df
sudo -n docker ps -a
sudo -n docker image ls -a
sudo -n docker volume ls
```

Preserve the named data volumes before more cleanup. A cloud-disk snapshot or a backup to a separate mounted disk/off-host destination avoids filling the already constrained root disk. A stopped PostgreSQL volume can be preserved as a physical copy, but a normal logical dump is preferable once the server has recovered. A same-disk copy is not protection against losing that disk.

Do not use volume pruning or Compose down -v for this recovery. Do not delete directories inside a PostgreSQL volume to free space. Retained Kafka and Prometheus data may be less essential than experiment history, but any deletion still needs an explicit retention decision and preservation of required evidence.

Build cache can be regenerated, whereas the application database cannot be reconstructed reliably from an incomplete Kafka retention window. Pruning build cache may force another long Rust compile. Inspect first and choose the smallest safe cleanup or add capacity before rebuilding. Reports and source are small compared with these Docker artifacts.

The local broker is configured for 24-hour retention, operational Prometheus for fifteen days, and course Prometheus for seven days. October metrics/events may be removed under retention when services resume. The saved benchmark JSON files remain the historical evidence even if an old dashboard time range no longer has live datasource samples.

Step 2 Choose the recovery mode

Objective	Compose selection	Kafka and notifications
Professor demo	compose.yml + compose.demo.yml	External experiment topic and external notification endpoint
Deterministic local development	compose.yml + compose.local.yml	Local experiment topic and notification double
Original instructor self-test	Base + local + course-test overlay, managed by loadtests.course	Local course-test-events and original Rust notification validator

These modes share the cec-temperature application project and database. Choose one intentionally. Starting local testing changes the application's topic and destination; it does not run an independent second application automatically.

If the exam is close, restore the core demo services first. They do not require compiling the Rust test tools or running local Kafka. Plan a separate local verification window before returning to demo mode.

Step 3 Rebuild and inspect the core state

Check that the existing credentials and database-password configuration are available. Reusing a PostgreSQL volume requires the same database credentials; setting a new POSTGRES_PASSWORD variable does not reset an existing database role.

The source already has a virtual environment. If it is still usable, the lightweight check is:

```
.venv/bin/python -m pytest -q -m 'not integration'
```

Prepare the narrow password file without printing its contents, then build the application/gateway:

```
.venv/bin/python scripts/prepare-monitoring-credentials.py group15-credentials
sudo -n docker compose -f compose.yml -f compose.demo.yml config --quiet
sudo -n docker compose -f compose.yml -f compose.demo.yml build data api
```

Start the database, cache, Data, executor, and API first, leaving event consumption and outbound delivery for the next step:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml up -d \
  postgres redis data executor api
sudo -n docker compose -f compose.yml -f compose.demo.yml ps
curl --fail http://127.0.0.1:3003/health
sudo -n docker compose -f compose.yml -f compose.demo.yml logs --tail 100 postgres data
```

Wait for health and review database recovery before relying on the data. Then inspect row counts and outbox status without dumping secret-bearing payloads:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml exec -T postgres \
  psql -U observability -d observability -c \
```

```
  "SELECT count(*) AS experiments FROM experiments;"
sudo -n docker compose -f compose.yml -f compose.demo.yml exec -T postgres \
  psql -U observability -d observability -c \
  "SELECT status, count(*) FROM notification_outbox GROUP BY status;"
```

Compare known experiment IDs/history against preserved evidence where possible. Take a logical database backup to the chosen external destination after successful recovery.

Inspect pending jobs before changing their notification destination. Because local and demo modes share the application database, an unfinished local test job is not automatically appropriate for the real endpoint. Preserve and diagnose unexpected jobs rather than blindly deleting history or starting delivery to a different validator.

Step 4 Verify external connectivity and start demo processing

The preflight is read-only: it neither publishes nor commits the live consumer's offsets.

```
.venv/bin/python scripts/check-course-kafka.py --topic experiment
```

A successful preflight establishes metadata access and a temporary consumer's assignment. It does not prove that the actual scheduler is running. After state and destination checks, start the actual processes:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml up -d scheduler notification
curl --fail http://127.0.0.1:8001/health
curl --fail http://127.0.0.1:8001/status
sudo -n docker compose -f compose.yml -f compose.demo.yml logs --tail 100 scheduler notification
```

Confirm the intended topic is experiment, the consumer group remains group15-temperature-demo-v1, and notifications go to the course endpoint. Do not change the group merely to clear a queue; a new group starts from earliest retained events and changes the checkpoint identity.

The historical course topic had sixteen partitions. Verify the current assignment rather than treating that old number as permanent. A growing queue immediately after recovery may represent catch-up; assess its trend, source timestamps, and notification age before interpreting fresh latency.

If TLS fails, distinguish port access, CA trust, hostname matching, keystore readability, authentication, and topic authorization. The demo overlay contains the known hostname exception; restore strict matching when valid public-name certificates are available. Do not print the token or keystore password during troubleshooting.

Step 5 Restore observation and check public access

When resources permit, start operational monitoring:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml \
  -f compose.monitoring.yml up -d prometheus node-exporter grafana
```

The public API is port 3003. Operational Prometheus is port 3008, used for the course's optional infrastructure metrics. Operational Grafana is loopback 3004. Access it through SSH forwarding from the laptop:

```
ssh -N -L 3004:127.0.0.1:3004 ubuntu@YOUR_VM_PUBLIC_IP
```

Use the existing SSH key/options appropriate to the VM. Verify API reachability from outside the VM as well as localhost; local health does not establish routing/firewall reachability for the professor.

For a known experiment, query both required endpoints:

```
curl --fail --get http://YOUR_VM_PUBLIC_IP:3003/temperature \
  --data-urlencode 'experiment-id=KNOWN_EXPERIMENT_ID' \
  --data-urlencode 'start-time=0' \
  --data-urlencode 'end-time=2000000000'
curl --fail --get http://YOUR_VM_PUBLIC_IP:3003/temperature/out-of-range \
  --data-urlencode 'experiment-id=KNOWN_EXPERIMENT_ID'
```

Use the course-hosted Grafana and a current workload/time range to observe the professor's results. Do not call an old pinned local test window current evidence.

Step 6 Revalidate locally in a separate test window

The source/configuration syntax checks succeeded during this report preparation, but running the full checks still requires recovered containers and enough space.

Use this window outside the professor's workload. Before switching modes, stop intake, let outstanding jobs drain to their current validator, and confirm that no pending jobs would be sent to the wrong destination.

For the deterministic local stack:

```
sudo -n docker compose -f compose.yml -f compose.local.yml up -d
TO_TEST_STACK=1 .venv/bin/python -m pytest -q
.venv/bin/python -m loadtests.stress --experiments 3 --sensors 3 \
  --duration 3 --http-rps 20 --output test-results/recovery-smoke.json
```

Integration tests create isolated test databases; they are not a substitute for verifying the retained production database. Do not benchmark concurrently with a build, test suite, or recovery exercise.

For the actual instructor tools:

```
.venv/bin/python -m loadtests.course run
.venv/bin/python -m loadtests.course run --experiments 100 --sensors 10 \
  --samples 60 --http-rps 100 --http-seconds 60
.venv/bin/python -m loadtests.course check
.venv/bin/python -m loadtests.course stop
```

The first missing Rust image may require a long compile and substantial disk space. Remaining intermediate images may help the cache, but should not be blindly retagged as completed runtimes; an intermediate can lack the final setup step.

The runner's prepare stage refreshes the generated local thirty-day JWT. It reuses suitable local certificate/key files and generates the workload/datasource configuration. The supplied external course token has a separate lifecycle.

Forward port 3009 to see the local grading dashboard. The added bottom Latest local test panel gives explicit runner status. Inspect actual counts and maximum latency as well as that status; apply the rubric's strict less-than-ten-second interpretation.

After local tests, stop the generators and explicitly restore the demo overlay. Confirm topic, destination, actual assignment, and public access again. Stopping the generators alone does not switch application configuration back to the course.

Prevent another full-disk incident

Set operational thresholds for free space and growth, with enough reserve for rebuilding images, database writes, and a test run. A proposed starting policy is warning below 25% free and an operational stop on optional builds/load below 15% free, then tune against measured growth. These are planning thresholds, not implemented alerts or a guarantee.

The existing alert rules cover latency, rejected events/jobs, and scrape health. They do not yet include filesystem-capacity alerts or external Alertmanager delivery. Node exporter exposes filesystem data that can support a new rule.

Periodically account for application history/indexes/outbox, Kafka retention, monitoring retention, logs, images, and build cache. A fixed Redis limit does not constrain these. Preserve historical experiment data through an agreed archive/export policy rather than silently deleting the user's requested trace.

Do not assume that removing containers frees their named data volumes. The November inspection demonstrated that distinction: the database and Kafka volumes alone retained several gigabytes after all containers were gone.

Final readiness check before the exam

A team member should be able to show, from a fresh record:

- A working public API and known correct history.
- The actual scheduler's current topic/group/assignment and a stable or draining queue.
- Correct external notification destination and an inspected outbox.
- Fresh nonzero workload validation, with required/received counts and strict latency.
- Variable-load evidence and honest capacity limits.
- Current disk headroom and a way to preserve/recover data.
- A current dashboard window, with local and hosted results clearly identified.
- All three members ready to explain both the design and one failure case.

The readiness audit is the starting point for recovery, not a completion record.